

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І.І.МЕЧНИКОВА
Факультет математики, фізики та інформаційних технологій
Кафедра математичного забезпечення комп'ютерних систем

КУРСОВА РОБОТА
з освітнього компоненту «Програмування»
на тему: СТВОРЕННЯ ІЄРАРХІЇ КЛАСІВ НА ТЕМУ «ГІПЕРМАРКЕТ»

Студента 1 курсу, денна форма навчання
спеціальність F7 «Комп'ютерна інженерія»

Турлакова Георгія Сергійовича

Керівник: к.т.н., доц. Шестопапов С.В.

Національна шкала: _____

Кількість балів: _____

ECTS: _____

Дата захисту: _____

Члени комісії _____ Антоненко О.С.
(підпис) (прізвище та ініціали)

_____ Шестопапов С.В.
(підпис) (прізвище та ініціали)

(підпис) (прізвище та ініціали)

ЗМІСТ

	стор.
ВСТУП.....	3
ЗАВДАННЯ.....	4
1. ТЕОРЕТИЧНА ЧАСТИНА.....	5
1.1 Поняття ООП.....	5
1.2 Опис об'єктної частини.....	6
2. ПРАКТИЧНА ЧАСТИНА.....	7
2.1 Опис класів.....	7
2.3 Інтерфейс.....	12
ВИСНОВКИ.....	18
СПИСОК ЛІТЕРАТУРИ.....	19

ВСТУП

Мета роботи – створити ієрархію класів на тему «Гіпермаркет» та застосувати її для розв’язання прикладної задачі, визначеної у варіанті завдання. Під час виконання роботи необхідно використовувати основні принципи об’єктно-орієнтованого програмування: інкапсуляцію, успадкування та поліморфізм. Згідно із завданням потрібно реалізувати систему взаємопов’язаних класів, що описують структуру та роботу гіпермаркету.

Необхідно створити конструктори класів, поля та методи для роботи з об’єктами. Поля класів повинні бути закритими, а доступ до них має здійснюватися через відповідні методи класу. У програмі потрібно реалізувати ієрархію класів на основі відношень успадкування та композиції. Для реалізації динамічного поліморфізму слід використовувати віртуальні методи, абстрактні класи та, за потреби, чисті віртуальні методи.

ЗАВДАННЯ

Варіант 21б. Створення ієрархії класів на тему «Гіпермаркет»

У роботі необхідно реалізувати класи, що описують структуру гіпермаркету, товари та працівників. Передбачити можливість роботи різними категоріями товарів, зберігання інформації про їх назву, ціну, кількість та інші характеристики. Також необхідно реалізувати взаємодію між класами, перевірку коректності даних та виконання основних операцій, пов'язаних із функціонуванням гіпермаркету.

1. ТЕОРЕТИЧНА ЧАСТИНА

1.1 Поняття ООП

Об'єктно-орієнтоване програмування – це метод програмування, заснований на поданні програми у вигляді сукупності взаємодіючих об'єктів, кожен з яких є екземпляром певного класу, а класи є членами певної ієрархії наслідування [1]. Програмісти спочатку пишуть клас, а на його основі при виконанні програми створюються конкретні об'єкти (екземпляри класів). На основі класів можна створювати нові, які розширюють базовий клас і таким чином створюється ієрархія класів.

Кожний стиль програмування має свою концептуальну базу. Для ОО стилю такою базою є об'єктна модель. Її побудова основана на використанні 3-х основних концепцій [2,3]:

- а) інкапсуляція;
- б) поліморфізм;
- с) наслідування.

Інкапсуляція – механізм, який поєднує дані та методи, що обробляють ці дані і захищає і те і інше від зовнішнього впливу або не вірного використання. Коли і методи і данні об'єднуються таким чином – створюється об'єкт [1].

Наслідування – процес, завдяки якому один об'єкт може придбати властивості іншого, тобто наслідувати властивість іншого об'єкту і додавати риси характерні тільки для нього самого [1].

Поліморфізм – властивість, яка дозволяє одне і те саме ім'я використовувати для вирішення декількох технічно різних задач, тобто основною метою поліморфізму є використання одного імені для задання загальних класу

дій. На практиці це значить спроможність об'єктів вибирати внутрішній метод або процедуру, виходячи із типу даних, прийнятих в повідомленні [1].

1.2 Опис об'єктної частини

Програма пропонує систему розподілу прав для трьох категорій користувачів: звичайний покупець, постійний клієнт та менеджер. Можливість виконання тих чи інших дій всередині програми безпосередньо залежить від поточної ролі користувача, перевірки його балансу (для здійснення покупок) або проходження процесу верифікації.

Звичайний покупець (Гість) та Постійний покупець взаємодіють з інтерфейсом магазину для перегляду асортименту товарів та здійснення покупок за допомогою методу Buy(). Звичайний клієнт купує товари за повною вартістю, якщо його баланс є достатнім. Постійний покупець авторизується за ПІБ і отримує додаткову перевагу у вигляді накопичувальної знижки, яка автоматично розраховується на основі суми його попередніх витрат і зменшує фінальну вартість товару під час оформлення покупки.

Менеджер (Адміністратор) отримує доступ до прихованого меню керування після введення спеціального символу та перевірки імені. У цьому режимі він може повністю контролювати роботу торгової точки: додавати нові товари на склад (враховуючи обмеження місткості), переглядати та редагувати базу постійних клієнтів (видаляти або реєструвати нових за умови виконання ліміту витрат), а також проводити за допомогою перегляду повної історії транзакцій магазину.

2. ПРАКТИЧНА ЧАСТИНА

2.1 Опис класів

Діаграму класів наведено в додатку А. Розглянемо більш детально структуру кожного класу.

- 1) Абстрактний клас «Goods»:
 - a) Приватні поля: `company`, `name`, `price`, `maxDiscount`.
 - b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
 - c) Методи:
 - Перевизначення `ToString` – повертає інформацію про об'єкт.
- 2) Абстрактний клас «HouseholdGoods», нащадок класу «Goods»:
 - a) Приватні поля: `power`, `cordLength`.
 - b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
 - c) Методи:
 - Перевизначення `ToString` – повертає інформацію про об'єкт.
- 3) Клас «VacuumCleaner», нащадок класу «HouseholdGoods»
 - a) Приватні поля `noiseLevel`
 - b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
 - c) Методи:
 - Перевизначення `ToString` – повертає інформацію про об'єкт.

- 4) Клас «Camera», нащадок класу «Goods»
 - a) Приватні поля: megapixels
 - b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
 - c) Методи:
 - Перевизначення ToString – повертає інформацію про об'єкт.
- 5) Клас «DSLR», нащадок класу «Camera»
 - a) Приватні поля: interchangeableLens, maxShutterSpeed.
 - b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
 - c) Методи:
 - Перевизначення ToString – повертає інформацію про об'єкт.
- 6) Клас «Computer», нащадок класу «Goods»
 - a) Приватні поля: processor, ram, gpu.
 - b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
 - c) Методи:
 - Перевизначення ToString – повертає інформацію про об'єкт.
- 7) Клас «Laptops», нащадок класу «Computer»
 - a) Приватні поля: weight, batteryLife.
 - b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
 - c) Методи:

- Перевизначення ToString – повертає інформацію про об'єкт.

8) Клас «Customer»

a) Приватні поля: balance

b) Конструктори:

- Конструктор без параметрів
- Конструктор з параметрами

c) Методи:

- Перевизначення ToString – повертає інформацію про об'єкт.
- Віртуальний метод Buy
- Віртуальний метод GetDiscount

9) Клас «RegularCustomer», нащадок класу «Customer»

a) Приватні поля: name, totalAmountSpent.

b) Конструктори:

- Конструктор без параметрів
- Конструктор з параметрами

c) Методи:

- Перевизначення ToString – повертає інформацію про об'єкт.
- Перевизначення методу Buy
- Перевизначення методу GetDiscount

10) Клас «Shop»

a) Приватні поля: address, managerName, storageCapacity.

b) Конструктори:

- Конструктор без параметрів
- Конструктор з параметрами

c) Методи:

- Перевизначення ToString – повертає інформацію про об'єкт.
- PrintGoods – роздруковує товари які є у певному магазині.

- AddPurchase – додає до історії покупок магазину куплений товар.
- CheckManagerName – перевіряє чи ведено ім'я менеджера магазину.
- PrintHistory – роздруковує історію покупок у магазині.

11) Клас «Purchase»:

- a) Приватні поля: buyer, product, price.
- b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
- c) Методи:
 - Перевизначення ToString – повертає інформацію про об'єкт.

12) Клас «Shop»

- a) Приватні поля: address, storageCapacity, managerName, goodsList, history.
- b) Конструктори:
 - Конструктор без параметрів
 - Конструктор з параметрами
- c) Лісти виключно для читання:
 - StandartCatalog
- d) Методи:
 - HasGoods – перевіряє чи є товари на складі.
 - PrintGoods – роздруковує товари з складу
 - AddPurchase – фіксує покупку у історії покупок
 - CheckManagerName – перевіряє ім'я менеджера
 - PrintHistory – роздруковує покупки які були здійсненні
 - Перевизначення ToString – повертає інформацію о об'єкті.

- AddDefaultGoods – додає товари за замовченням із списку стандартних товарів.

13) Клас «Session»

- a) Приватні поля: currentShop, currentCustomer, securityLevel, shopList, customerList.
- b) Лісти виключно для читання:
 - StandartCustomer
 - StandartShop
- c) Методи:
 - Reset – обнулює усі поля класа.
 - AddStandartCustomers – додає випадкових клієнтів за замовченням із списку стандартних клієнтів
 - AddStandartShops – додає випадкові магазини із списком стандартних магазинів.
 - Reset – виставляє параметри сесії за замовченням

14) Клас «Menu»

- a) Методи:
 - ShowShopSelectMenu – відображає вікно вибору магазину
 - ShowCustomerSelectMenu – відображає вікно авторизації
 - ShowMainMenu – відображає варіант головного вікна для гостя або постійного клієнта
 - ShowProducts – відображає вікно вибору товару
 - ShowInfo – відображає вікно інформації магазину
 - ShowCustomersManager – відображає варіант головного вікна для менеджера
 - ShowAddProduct – відображає вікно для додавання товарів.
 - ShowHistory – відображає вікно історії покупок.

2.3 Інтерфейс

Готовий проєкт розміщено в [4]. Під час запуску програми користувач бачить вступне вікно (Рис. 3.1).

```
Виберіть магазин:
№ | Адреса
---+-----
01 | 123 Main St
02 | 231 Oak Ave
03 | 39 Oak St

0 Вихід
```

Рисунок 3.1 – початкове вікно

Користувач потрапляє сюди коли запускає програму, або виходить з магазину для вибору іншого магазину з цієї мережі. Після того як користувач введе номер магазину до якого хоче зайти, йому буде запропоновано пройти верифікацію (Рис.3.2).

```
Доброго Дня! Ви постійний клієнт?
у - так, н - ні
```

Рисунок 3.2 – Вікно верифікації

Тут у користувача є 3 варіанти: написати «n», «у», «а».

```
Доброго Дня! Ви постійний клієнт?  
у - так, п - ні  
п  
Введіть свій баланс  
12000_
```

Рисунок 3.3 – Вікно для звичайного покупця.

У випадку якщо користувач напише «п» (приклад вікна Рис.3.3) користувачу буде запропоновано вести свій баланс, після чого він попаде до головного вікна (Рис. 3.5) як гість.

```
Доброго Дня! Ви постійний клієнт?  
у - так, п - ні  
у  
Введіть своє ПІБ  
Ivan  
Введіть свій баланс  
120000_
```

Рисунок 3.4 – Вікно для постійного покупця.

У випадку якщо користувач напише «у» (приклад вікна Рис.3.4) користувачу буде запропоновано вести свої ім'я і якщо вони співпадатиме з ім'ям якогось користувача то йому також буде запропоновано ввести баланс, але він попаде до головного вікна (Рис. 3.5) вже як постійний клієнт.

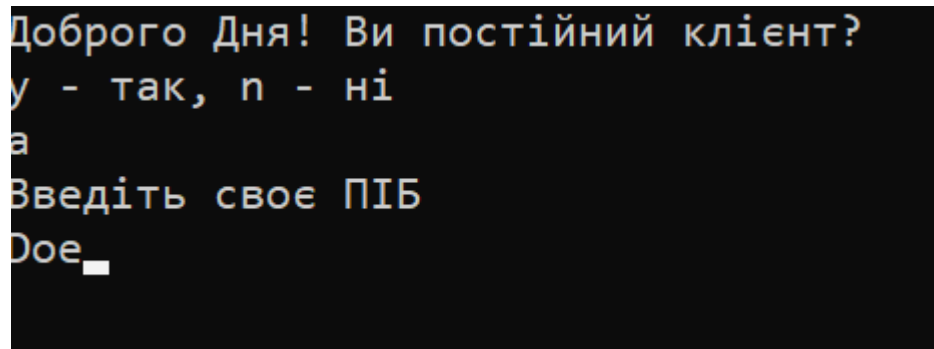


Рисунок 3.5 – Вікно для менеджера

У випадку якщо користувач напише «а» (літра «а» це скорочення від «admin»), то з'явиться приховане вікно для верифікації менеджера (приклад вікна Рис. 3.5). Якщо введене ім'я співпадатиме з ім'ям менеджера конкретного магазину, то він буде авторизований як менеджер, і в нього з'являться нові приховані розділи (Рис. 3.9), призначені виключно для менеджерів.

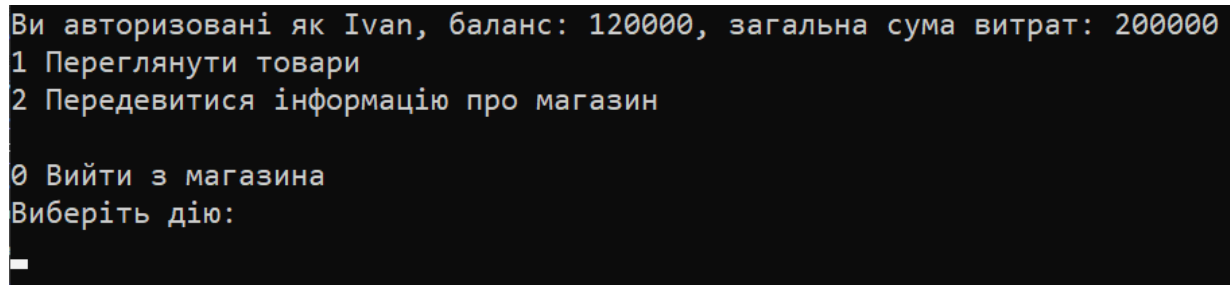


Рисунок 3.6 – Головне вікно (Не менеджер)

Після того як користувач пройде верифікацію він потрапить до головного вікна. Воно відрізняється від типу користувача.

Тут користувач може обрати:

- а) переглянути товари які є у цьому магазині (Рис 3.7);
- б) переглянути інформація о магазині;
- с) вийти з магазину та піти у інший.

```

Товари в магазині за адресою: 123 Main St
1 Пилосос: Компанія: BOSCH | Назва: BGC05AAA1 | Ціна: 3699 грн | Макс. знижка: 10% | Потужність | Довжина шнура: 6 м | Рівень шуму: 78 дБ
2 Камера: Компанія: Canon | Назва: PowerShots SX40 HS | Ціна: 15999 грн | Макс. знижка: 15% | Об'єм зйомки: 12 МП
Щоб купити товар, введіть його номер
Для виходу введіть 0

```

Рисунок 3.7 – Перегляд товарів

Тут користувач може передивитися увесь товар що є у магазині та обрати що він хоче купити. Для того щоб щось купити треба написати номер товару. Якщо буде вистачати грошей, то повернеться повідомлення про те що товар куплений. Також він зникне з цього списку і з'явиться нове поле у історії покупок.

Якщо грошей буде не вистачати, то програма надрукує помилку, та знов повернетесь на це вікно.

Для того щоб повернутись до головного вікна треба написати «0»

```

Магазин: Адреса: 123 Main St
Ємність складу: 10 одиниць
Кількість товарів: 2
Номер гарячої лінії: (049) 949-23-32

Графік роботи:
Пн-Пт 9:00-20:00
Сб 10:00-18:00
Неділя вихідний
Натисніть будь-яку клавішу для продовження

```

Рисунок 3.8 – Інформація магазину

У цьому вікні користувач може передивитися інформацію о магазині: Адреса, Ємність складу, Кількість товару. Номер гарячої лінії, та графік роботи.

Якщо користувач зайдє як менеджер, для нього буде доступні нові розділи.

```

Ви авторизовані як менеджер Doe.
1 Переглянути товари
2 Передевитися інформацію про магазин
3 Переглянути постійних покупців
4 Додати товар
5 Передевитися історію покупок.

0 Вийти з магазина

```

Рисунок 3.9 – Головне вікно (Менеджер)

Тут додається 3 розділи доступні виключно для Менеджерів:

- a) перегляд списку постійних покупців;
- b) додати товар;
- c) передавитися історію покупок.

```

Список постійних покупців
1 Постійний покупець: Ivan, Баланс: 0, Витрачено усього: 200000
2 Постійний покупець: Peter, Баланс: 0, Витрачено усього: 52000
3 Постійний покупець: deBug, Баланс: 1000000, Витрачено усього: 10000000

1 Додати постійного клієнта
2 Видалити постійного клієнта

0 Назад
Виберіть дію:
_

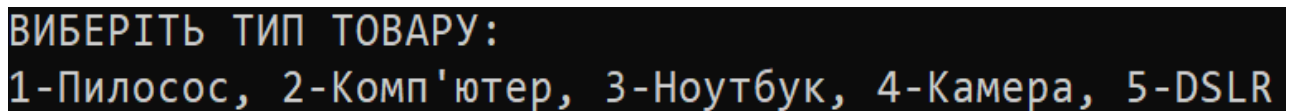
```

Рисунок 3.10 – Вікно списку постійних покупців

Тут виводиться список усіх постійних покупців мережі магазинів. Для того щоб додати або видалити покупця треба обрати потрібну дію.

У випадку видалення треба бути ввести номер клієнта після чого він зникне з бази.

У випадку додавання треба ввести: Ім'я покупця, та скільки він витратив грошей. При цьому якщо загальних витрат буде менше ніж на 50.000, буде повернута помилка з повідомленням що для створення нового постійного покупця він повинен витрати хоча б 50.000.

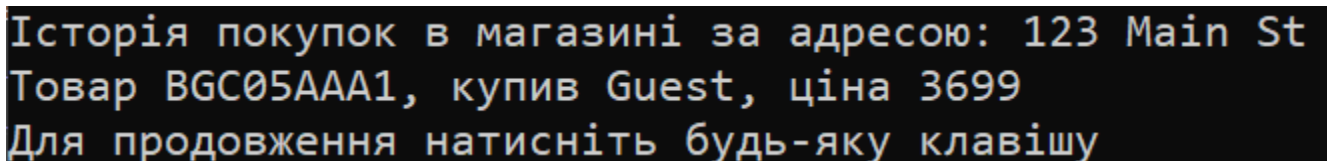


```
ВИБЕРІТЬ ТИП ТОВАРУ:  
1-Пилосос, 2-Комп'ютер, 3-Ноутбук, 4-Камера, 5-DSLR
```

Рисунок 3.11 – Вікно створення товару.

Тут пропонується вибрати один з п'яти конструкторів та ввести усі данні для певного товару.

Якщо склад магазину вже повний, то створити новий товар буде не можливо.



```
Історія покупок в магазині за адресою: 123 Main St  
Товар BGC05AAA1, купив Guest, ціна 3699  
Для продовження натисніть будь-яку клавішу
```

Рисунок 3.12 – Вікно історії покупок.

Тут друкується уся інформація по товарам які були куплені у певному магазині.

Якщо користувач повернеться до початкового вікна з головного вікна, і вибере інший магазин. То йому доведеться знову проходити верифікацію.

ВИСНОВКИ

У ході виконання курсової роботи було розроблено програму на мові C# з використанням принципів об'єктно-орієнтованого програмування на тему «Гіпермаркет». У процесі роботи було створено ієрархію класів для представлення товарів, покупців, покупок та магазинів, а також реалізовано взаємодію між ними.

Під час розробки програми були використані основні принципи ООП: інкапсуляція, наслідування та поліморфізм. Для кожного класу були створені відповідні поля, властивості, конструктори та методи. Також було реалізовано систему авторизації менеджера з прихованими функціями доступу, історію покупок, перевірку прав доступу та графік роботи магазину залежно від дня тижня і часу.

У результаті виконання курсової роботи були закріплені практичні навички програмування на C#, роботи з класами, колекціями, методами, властивостями та консольним інтерфейсом. Розроблена програма демонструє можливість створення повноцінної системи управління гіпермаркетом засобами об'єктно-орієнтованого програмування.

СПИСОК ЛІТЕРАТУРИ

1. Web Library, Основні поняття ООП. [Електронне видання] / Тернопіль 2026. – Режим доступу: <http://weblib.com.ua/blog/osnovni-ponyattya-oor>
2. W3School, C# OOP (Object-Oriented Programming). Режим доступу: https://www.w3schools.com/cs/cs_oor.php
3. Є.О. Віктор, О.А. Геренко, І.М. Шпінарева, Практикум з курсу Об'єктно-орієнтоване програмування мовою C#. [Електронне видання] / Одеса, 2026. – Режим доступу: <https://oor-practicum-csharp-851f60.gitlab.io/>
4. Репозиторій GitHub на курсовий – Режим доступу: <https://github.com/Offiks/Kursach>

Додаток А. Діаграма класів

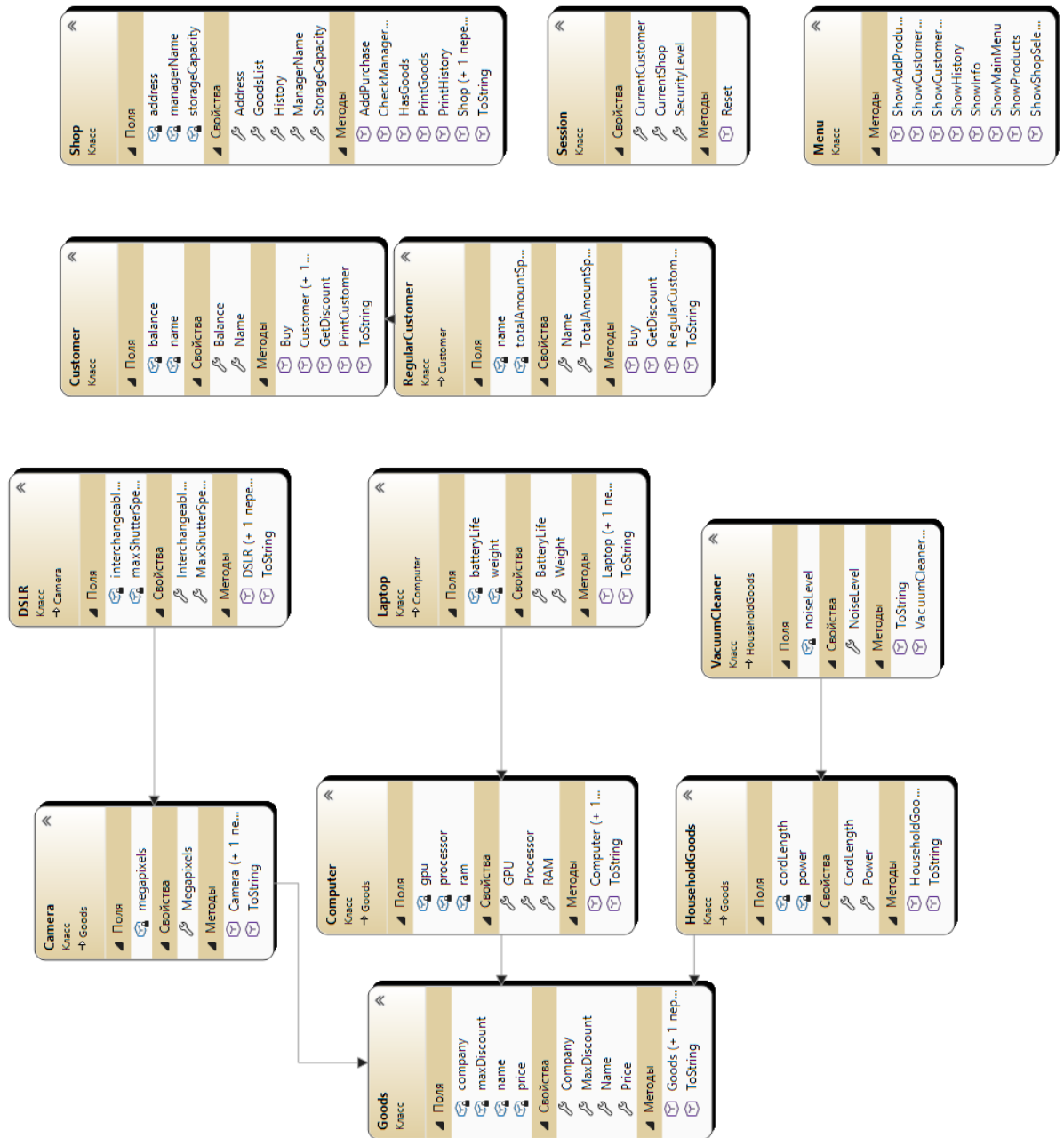


Рисунок А1. – Діаграма класів

Додаток Б. Код класу

```
using System;
namespace Kursach.Classes
{
    public class Camera : Goods
    {
        private int megapixels;

        public Camera() : this("N/A", "N/A", 0, 0, 0) { }

        public Camera(string company, string name, int price, int maxDiscount, int
megapixels)
            : base(company, name, price, maxDiscount)
        {
            Megapixels = megapixels;
        }
        public int Megapixels
        {
            get { return megapixels; }
            set
            {
                if (value < 0)
                    throw new ArgumentException("Megapixels cannot be negative");
                megapixels = value;
            }
        }

        public override string ToString()
        {
            return $"Камера: Компанія: {Company} | Назва: {Name} | Ціна: {Price} грн |
Макс. знижка: {MaxDiscount}% | Мегапікселі: {Megapixels} МП";
        }
    }
}
```

Лістинг Б.1 – Клас Camera

```
using System;

namespace Kursach.Classes
{
    public class Computer : Goods
    {
        private string processor;
        private int ram;
        private string gpu;

        public Computer() : this("N/A", "N/A", 0, 0, "N/A", 0, "N/A") { }
    }
}
```

Лістинг Б.2 – Клас Computer, аркуш 1

```

public Computer(string company, string name, int price, int maxDiscount, string processor,
int ram, string gpu)
    : base(company, name, price, maxDiscount)
{
    Processor = processor;
    RAM = ram;
    GPU = gpu;
}
public string Processor
{
    get { return processor; }
    set
    {
        if (string.IsNullOrEmpty(value))
            throw new ArgumentException("Processor cannot be empty");
        processor = value;
    }
}
public int RAM
{
    get { return ram; }
    set
    {
        if (value < 0)
            throw new ArgumentException("RAM cannot be negative");
        ram = value;
    }
}
public string GPU
{
    get { return gpu; }
    set
    {
        if (string.IsNullOrEmpty(value))
            throw new ArgumentException("GPU cannot be empty");
        gpu = value;
    }
}
public override string ToString()
{
    return $"Комп'ютер: Компанія: {Company} | Назва: {Name} | Ціна: {Price} грн |
Макс. знижка: {MaxDiscount}% | Процесор: {Processor} | ОЗУ: {RAM} ГБ | Відеокарта: {GPU}";
}
}
}

```

ЛІСТИНГ Б.2 – Клас Computer, аркуш 2

```

using System;
using System.Collections.Generic;
using System.Net;
using System.Xml.Linq;

namespace Kursach
{
    public class Customer
    {
        private int balance;
        private readonly string name = "Guest";

        public Customer() : this(0, "Guest") { }
    }
}

```

```

public Customer(int balance, string name = "Guest")
{
    Balance = balance;
}

public int Balance
{
    get { return balance; }
    set
    {
        if (value < 0)
            throw new ArgumentException("Balance cannot be negative");
        balance = value;
    }
}

public string Name
{
    get { return name; }
}

public static void PrintCustomer(List<Customer> list)
{
    Console.WriteLine($"Список постійних покупців");
    if (list == null)
    {
        Console.WriteLine("Список постійних покупців порожній.");
        Console.WriteLine("Для продовження натисніть будь яку кнопку");
        Console.ReadKey();
    }
    else
    {
        for (int i = 1; i < list.Count; i++)
        {
            Console.WriteLine($"{i} {list[i]}");
        }
    }
}

public virtual (string Name, int PersonalDiscount) GetDiscount(Goods goods)
{
    return (string.Empty, 0);
}

public virtual bool Buy(Goods goods)
{
    var (customerName, discount) = GetDiscount(goods);

    int finalPrice = goods.Price - (goods.Price * discount / 100);
    if (Balance >= finalPrice)
    {
        Balance -= finalPrice;
        return true;
    }
    else
    {
        return false;
    }
}

```

```

public override string ToString()
{
    return $"Покупець: Баланс: {Balance}";
}
}

```

Лістинг Б.3 – Клас Customer, аркуш 2

```

using System;

namespace Kursach.Classes
{
    public class DSLR : Camera
    {
        private bool interchangeableLens;
        private int maxShutterSpeed;

        public DSLR() : this("N/A", "N/A", 0, 0, 0, false, 0) { }

        public DSLR(string company, string name, int price, int maxDiscount, int
megapixels, bool interchangeableLens, int maxShutterSpeed)
            : base(company, name, price, maxDiscount, megapixels)
        {
            InterchangeableLens = interchangeableLens;
            MaxShutterSpeed = maxShutterSpeed;
        }

        public bool InterchangeableLens
        {
            get { return interchangeableLens; }
            set { interchangeableLens = value; }
        }

        public int MaxShutterSpeed
        {
            get { return maxShutterSpeed; }
            set
            {
                if (value < 0)
                    throw new ArgumentException("Max shutter speed cannot be negative");
                maxShutterSpeed = value;
            }
        }

        public override string ToString()
        {
            return $"DSLR: Компанія: {Company} | Назва: {Name} | Ціна: {Price} грн | Макс.
знижка: {MaxDiscount}% | Мегапікселі: {Megapixels} МП | Змінний об'єктив:
{InterchangeableLens} | Макс. витримка: {MaxShutterSpeed} с";
        }
    }
}

```

Лістинг Б.4 – Клас DSLR


```

using System;

namespace Kursach
{
    public abstract class Goods
    {
        private string company;
        private string name;
        private int price;
        private int maxDiscount;

        public Goods() : this ("N/A", "N/A", 0, 0) { }

        public Goods(string company, string name, int price, int maxDiscount)
        {
            Company = company;
            Name = name;
            Price = price;
            MaxDiscount = maxDiscount;
        }

        public string Company
        {
            get { return company; }
            set
            {
                if (string.IsNullOrEmpty(value))
                    throw new ArgumentException("Company cannot be empty");
                company = value;
            }
        }

        public string Name
        {
            get { return name; }
            set
            {
                if (string.IsNullOrEmpty(value))
                    throw new ArgumentException("Name cannot be empty");
                name = value;
            }
        }

        public int Price
        {
            get { return price; }
            set
            {
                if (value < 0)
                    throw new ArgumentException("Price cannot be negative");
                price = value;
            }
        }
    }
}

```

Лістинг Б.5 – Клас Goods, аркуш 1

```

public int MaxDiscount
{
    get { return maxDiscount; }
    set
    {
        if (value < 0 || value > 100)
            throw new ArgumentException("Discount must be 0-100");
        maxDiscount = value;
    }
}

public override string ToString()
{
    return $"Товар: Компанія: {Company} | Назва: {Name} | Ціна: {Price} грн | Макс.
знижка: {MaxDiscount}%";
}
}
}

```

Лістинг Б.5 – Клас HouseholdGoods, аркуш 2

```

using System;

namespace Kursach
{
    public abstract class HouseholdGoods : Goods
    {
        private int power;
        private double cordLength;

        public HouseholdGoods() : this ("N/A", "N/A", 0, 0, 0, 0) { }

        public HouseholdGoods(string company, string name, int price, int maxDiscount,
double cordLength, int power)
            : base(company, name, price, maxDiscount)
        {
            Power = power;
            CordLength = cordLength;
        }

        public int Power
        {
            get { return power; }
            set
            {
                if (value < 0)
                    throw new ArgumentException("Power cannot be negative");
                power = value;
            }
        }

        public double CordLength
        {
            get { return cordLength; }
            set
            {
                if (value < 0)
                    throw new ArgumentException("Cord length cannot be negative");
                cordLength = value;
            }
        }
    }
}

```

Лістинг Б.6 – Клас HouseholdGoods, аркуш 1

```

public override string ToString()
{
    return $"Побутовий товар: Компанія: {Company} | Назва: {Name} | Ціна: {Price}
грн | Макс. знижка: {MaxDiscount}% | Потужність: {Power} Вт | Довжина шнура: {CordLength}
м";
}
}
}

```

Лістинг Б.6 – Клас HouseholdGoods, аркуш 2

```

using System;

namespace Kursach.Classes
{
    public class Laptop : Computer
    {
        private double weight;
        private double batteryLife;

        public Laptop() : this("N/A", "N/A", 0, 0, "N/A", 0, "N/A", 0, 0) { }
        public Laptop(string company, string name, int price, int maxDiscount, string
processor, int ram, string gpu, double weight, double batteryLife)
            : base(company, name, price, maxDiscount, processor, ram, gpu)
        {
            Weight = weight;
            BatteryLife = batteryLife;
        }
        public double Weight
        {
            get { return weight; }
            set
            {
                if (value < 0)
                    throw new ArgumentException("Weight cannot be negative");
                weight = value;
            }
        }
        public double BatteryLife
        {
            get { return batteryLife; }
            set
            {
                if (value < 0)
                    throw new ArgumentException("Battery life cannot be negative");
                batteryLife = value;
            }
        }
        public override string ToString()
        {
            return $"Ноутбук: Компанія: {Company} | Назва: {Name} | Ціна: {Price} грн |
Макс. знижка: {MaxDiscount}% | Процесор: {Processor} | ОЗУ: {RAM} ГБ | Відеокарта: {GPU} |
Вага: {Weight} кг | Час роботи від батареї: {BatteryLife} год";
        }
    }
}

```

Лістинг Б.7 – Клас Laptop, аркуш 2

```

using System;
using System.Collections.Generic;
using System.Runtime.Remoting.Messaging;

namespace Kursach.Classes
{
    public class Menu
    {
        public static Shop ShowShopSelectMenu(List<Shop> shopList)
        {
            while (true)
            {
                Console.Clear();

                Console.WriteLine("Виберіть магазин:");
                Console.WriteLine("% | Адреса");
                Console.WriteLine("----+-----");

                for (int i = 0; i < shopList.Count; i++)
                {
                    Console.WriteLine($"{i + 1:D2} | {shopList[i].Address}");
                }

                Console.WriteLine("\n0 Вихід");

                if (!int.TryParse(Console.ReadLine(), out int shopIndex))
                {
                    Console.WriteLine("Некоректне значення!");
                    Console.WriteLine("Натисніть будь-яку клавішу...");
                    Console.ReadKey();
                    continue;
                }

                if (shopIndex == 0)
                {
                    return null;
                }

                if (shopIndex < 1 || shopIndex > shopList.Count)
                {
                    Console.WriteLine("Такого магазину не існує!");
                    Console.WriteLine("Натисніть будь-яку клавішу...");
                    Console.ReadKey();

                    continue;
                }

                return shopList[shopIndex - 1];
            }
        }

        public static (int securityLevel, Customer customer)
        ShowCustomerSelectMenu(List<Customer> CustomerList, Shop currentShop)
        {
            while (true)
            {
                int currentSecurityLevel = 0;
                Customer currentSessionCustomer = null;
            }
        }
    }
}

```

Лістинг Б.8 – Клас Menu, аркуш 1

```

Console.Clear();
Console.WriteLine("Доброго Дня! Ви постійний клієнт?");
Console.WriteLine("y – так, n – ні");
Console.Write("Ваш вибір: ");

string input = Console.ReadLine();
switch (input)
{
    case "n":
    {
        Console.Write("Введіть свій баланс: ");

        if (!int.TryParse(Console.ReadLine(), out int CustomerBalance))
        {
            Console.WriteLine("Некоректне значення!");
            Console.WriteLine("Натисніть будь-яку клавішу...");
            Console.ReadKey();
            continue;
        }

        CustomerList[0].Balance = CustomerBalance;
        currentSecurityLevel = 0;

        return (currentSecurityLevel, CustomerList[0]);
    }
    case "y":
    {
        Console.Write("Введіть своє ПІБ: ");
        string cheakName = Console.ReadLine();
        if (string.IsNullOrEmpty(cheakName))
        {
            Console.WriteLine("Ім'я не може бути порожнім!");
            Console.WriteLine("Натисніть будь-яку клавішу щоб
продовжити");

            Console.ReadKey();
            continue;
        }

        Customer foundCustomer = null;

        for (int i = 1; i < CustomerList.Count; i++)
        {
            if (CustomerList[i] is RegularCustomer regularCustomer &&
                regularCustomer.Name == cheakName)
            {
                foundCustomer = CustomerList[i];
                break;
            }
        }
        if (foundCustomer == null)
        {
            Console.WriteLine("Клієнт з таким ім'ям не знайдений!");
            Console.WriteLine("Натисніть будь-яку клавішу щоб
продовжити");

            Console.ReadKey();
            continue;
        }
        Console.WriteLine($"Клієнта знайдено");
        Console.Write("Введіть свій баланс: ");
    }
}

```

```

        if (!int.TryParse(Console.ReadLine(), out int customerBalance))
        {
            Console.WriteLine("Некоректне значення!");
            Console.ReadKey();
            continue;
        }
        foundCustomer.Balance = customerBalance;
        currentSecurityLevel = 1;

        return (currentSecurityLevel, foundCustomer);
    }
    case "a":
    {
        Console.WriteLine("Введіть своє ПІБ");
        string checkName = Console.ReadLine();

        if (string.IsNullOrEmpty(checkName))
        {
            Console.WriteLine("Ім'я не може бути порожнім!");
            Console.WriteLine("Натисніть будь-яку клавішу щоб
продовжити");
            Console.ReadKey();
            continue;
        }

        if (currentShop.CheckManagerName(checkName))
        {
            currentSessionCustomer = new RegularCustomer(0,
currentShop.ManagerName, 0);
            currentSecurityLevel = 2;

            return (currentSecurityLevel, currentSessionCustomer);
        }
        else
        {
            Console.WriteLine("Невірне ім'я менеджера!");
            Console.ReadKey();
            continue;
        }
    }
    default:
    {
        Console.WriteLine("Невірне значення. Для продовження натисніть
будь яку клавішу.");
        Console.ReadKey();
        continue;
    }
}
}

}
}
public static int ShowMainMenu(Shop currentShop, int currentSecurityLevel, Customer
currentSessionCustomer)
{
    while (true)
    {
        Console.Clear();
        if (currentSecurityLevel == 0)
        {

```

Лістинг Б.8 – Клас Menu, аркуш 3

```

        Console.WriteLine($"Ви авторизовані як гість, баланс:
{currentSessionCustomer.Balance}");
    }
    else if (currentSecurityLevel == 1 && currentSessionCustomer is
RegularCustomer rc)
    {
        Console.WriteLine($"Ви авторизовані як {rc.Name}, баланс: {rc.Balance},
загальна сума витрат: {rc.TotalAmountSpent}");
    }
    else if (currentSecurityLevel == 2)
    {
        Console.WriteLine($"Ви авторизовані як менеджер
{currentShop.ManagerName}.");
    }

    if (currentSecurityLevel <= 1)
    {
        Console.WriteLine("1 Переглянути товари\n2 Передевитися інформацію про
магазин\n\n0 Вийти з магазину");
    }
    else
    {
        Console.WriteLine("1 Переглянути товари\n2 Передевитися інформацію про
магазин\n3 Переглянути постійних покупців\n4 Додати товар\n5 Передевитися історію
покупок.\n\n0 Вийти з магазину");
    }
    Console.Write("Оберіть дію: ");
    int choicePage;
    if (!int.TryParse(Console.ReadLine(), out choicePage))
    {
        Console.WriteLine("Некоректне значення!");
        Console.WriteLine("Натисніть будь-яку клавішу щоб продовжити");
        Console.ReadKey();
        continue;
    }
    if (choicePage < 0 || choicePage > 5)
    {
        Console.WriteLine("Невірний вибір. Натисніть будь-яку клавішу для
продовження");
        Console.ReadKey();
        continue;
    }
    return choicePage;
}
}
public static void ShowProducts(Shop currentShop, Customer currentSessionCustomer)
{
    while (true)
    {
        Console.Clear();
        currentShop.PrintGoods();
        if (!currentShop.HasGoods()) break;

        if (!int.TryParse(Console.ReadLine(), out int choiceGoods))
        {
            Console.WriteLine("Помилка вводу. Введіть коректне число.");
            Console.ReadKey();
            continue;
        }
        if (choiceGoods == 0) break;
    }
}

```

```

        if (choiceGoods < 1 || choiceGoods > currentShop.GoodsList.Count)
        {
            Console.WriteLine("Невірний вибір. Натисніть будь-яку клавішу для
продовження");
            Console.ReadKey();
            continue;
        }
        if (currentSessionCustomer.Buy(currentShop.GoodsList[choiceGoods - 1]))
        {
            currentShop.AddPurchase(currentSessionCustomer,
currentShop.GoodsList[choiceGoods - 1], currentShop.GoodsList[choiceGoods - 1].Price);
            currentShop.GoodsList.RemoveAt(choiceGoods - 1);
            Console.WriteLine("Покупка успішна!");
            Console.WriteLine("Натисніть будь-яку клавішу для продовження");
            Console.ReadKey();
        }
        else
        {
            Console.WriteLine("Недостатньо коштів для покупки цього товару!");
            Console.WriteLine("Натисніть будь-яку клавішу для продовження");
            Console.ReadKey();
            continue;
        }
    }
}

public static void ShowInfo(Shop currentShop, int securityLevel)
{
    Console.Clear();
    Console.WriteLine(currentShop.ToString(securityLevel));
    Console.WriteLine("Натисніть будь-яку клавішу для продовження");
    Console.ReadKey();
}

public static void ShowCustomersManager(Shop currentShop, List<Customer>
CustomerList)
{
    while (true)
    {
        Console.Clear();
        Customer.PrintCustomer(CustomerList);

        Console.WriteLine("\n1 Додати постійного клієнта\n2 Видалити постійного
клієнта\n\n0 Назад");
        Console.Write("Виберіть дію: ");

        if (!int.TryParse(Console.ReadLine(), out int choice))
        {
            Console.WriteLine("Помилка вводу. Введіть коректне число.");
            Console.ReadKey();
            continue;
        }
        if (choice == 0) break;
        switch (choice)
        {
            case 1:
            {
                Console.Clear();
                Console.WriteLine("Введіть ім'я нового постійного клієнта");
                string name = Console.ReadLine();

                Console.WriteLine("Введіть загальну суму витрат нового
постійного клієнта");

```



```

if (!int.TryParse(Console.ReadLine(), out int totalAmountSpent))
{
    Console.WriteLine("Помилка вводу. Введіть коректне
число.");
    Console.ReadKey();
    continue;
}
CustomerList.Add(new RegularCustomer(0, name,
totalAmountSpent));

Console.WriteLine("Постійного клієнта додано!");
Console.WriteLine("Натисніть будь-яку клавішу для
продовження");
Console.ReadKey();
break;
}
case 2:
{
    Console.WriteLine("Введіть номер постійного клієнта, якого
хочете видалити");
    int num = int.Parse(Console.ReadLine());
    CustomerList.RemoveAt(num);

    Console.WriteLine("Постійного клієнта видалено!");
    Console.WriteLine("Натисніть будь-яку клавішу для
продовження");
    Console.ReadKey();
    break;
}
}
}
}
public static void ShowAddProduct(Shop currentShop)
{
    while (true)
    {
        if (currentShop.GoodsList.Count >= currentShop.StorageCapacity)
        {
            Console.WriteLine("Склад повний! Для початку треба щоб хтось купив
товар, або видалити товар");
            Console.WriteLine("Натисніть будь-яку клавішу для продовження");
            Console.ReadKey();
            break;
        }
        try
        {
            Console.Clear();
            Console.WriteLine("Виберіть тип товару:");
            Console.WriteLine("1-Пилосос, 2-Комп'ютер, 3-Ноутбук, 4-Камера, 5-
DSLR");
            int goodTypeChosen = int.Parse(Console.ReadLine());

            Console.Write("Компанія: ");
            string comp = Console.ReadLine();

            Console.Write("Модель: ");
            string model = Console.ReadLine();

            Console.Write("Ціна: ");

```

```

int pr = int.Parse(Console.ReadLine());

Console.Write("Макс. знижка %: ");
int ds = int.Parse(Console.ReadLine());

switch (goodTypeChoosen)
{
    case 1:
        Console.Write("Шум(дБ): ");
        int noise = int.Parse(Console.ReadLine());

        Console.Write("Кабель(м): ");
        int len = int.Parse(Console.ReadLine());

        Console.Write("Потужність: ");
        int pwr = int.Parse(Console.ReadLine());

        currentShop.GoodsList.Add(new VacuumCleaner(comp, model, pr,
ds, noise, len, pwr));
        break;

    case 2:
        Console.Write("Процесор: ");
        string cpu = Console.ReadLine();

        Console.Write("ОЗП: ");
        int ram = int.Parse(Console.ReadLine());

        Console.Write("Відеокарта: ");
        string gpu = Console.ReadLine();

        currentShop.GoodsList.Add(new Computer(comp, model, pr, ds,
cpu, ram, gpu));
        break;

    case 3:
        Console.Write("Процесор: ");
        string Cpu = Console.ReadLine();

        Console.Write("ОЗП: ");
        int Ram = int.Parse(Console.ReadLine());

        Console.Write("Відеокарта: ");
        string Gpu = Console.ReadLine();

        Console.Write("Вага: ");
        double weight = double.Parse(Console.ReadLine());

        Console.Write("Батарея(год): ");
        int battery = int.Parse(Console.ReadLine());

        currentShop.GoodsList.Add(new Laptop(comp, model, pr, ds, Cpu,
Ram, Gpu, weight, battery));
        break;

    case 4:
        Console.Write("Мп: ");
        int mp = int.Parse(Console.ReadLine());

```

Лістинг Б.8 – Клас Menu, аркуш 7

```

        currentShop.GoodsList.Add(new Camera(comp, model, pr, ds, mp));
        break;
    case 5:
        Console.WriteLine("Мп: ");
        int d_mp = int.Parse(Console.ReadLine());

        Console.WriteLine("Змінний об'єктив (true/false): ");
        bool lens = bool.Parse(Console.ReadLine());

        Console.WriteLine("Витримка: ");
        int sh = int.Parse(Console.ReadLine());

        currentShop.GoodsList.Add(new DSLR(comp, model, pr, ds, d_mp,
lens, sh));
        break;
    }
}
catch (Exception ex)
{
    Console.WriteLine(ex.Message);
    Console.ReadKey();
    break;
}
Console.WriteLine("Товар додано!");
Console.WriteLine("Натисніть будь-яку клавішу для продовження");
Console.ReadKey();
break;
}
}
public static void ShowHistory(Shop currentShop)
{
    while (true)
    {
        Console.Clear();
        currentShop.PrintHistory();
        Console.WriteLine("Для продовження натисніть будь-яку клавішу");
        Console.ReadKey();
        break;
    }
}
}
}
}

```

Лістинг Б.8 – Клас Menu, аркуш 8

```

using Kursach;
using System;

public class Purchase
{
    private Customer buyer;
    private Goods product;
    private int price;

```

Лістинг Б.9 – Клас Purchase, аркуш 1

```

public Purchase(Customer buyer, Goods product, int price)
{
    Buyer = buyer;
    Product = product;
    Price = price;
}

public Customer Buyer
{
    get { return buyer; }
    set
    {
        if (value != null)
        {
            buyer = value;
        }
        else
        {
            Console.WriteLine("Покупець не може бути порожнім.");
        }
    }
}

public Goods Product
{
    get { return product; }
    set
    {
        if (value != null)
        {
            product = value;
        }
        else
        {
            Console.WriteLine("Товар не може бути порожнім.");
        }
    }
}

public int Price
{
    get { return price; }
    set
    {
        if (value > 0)
        {
            price = value;
        }
        else
        {
            Console.WriteLine("Ціна повинна бути більше 0.");
        }
    }
}

public override string ToString()
{
    return $"Товар {Product.Name}, купив {Buyer.Name}, ціна {Price}";
}
}

```

```

using System;
using System.Net;

namespace Kursach
{
    public class RegularCustomer : Customer
    {
        private string name;
        private int totalAmountSpent;

        public RegularCustomer() : this(0, "N/A", 0) { }
        public RegularCustomer(int balance, string name, int total)
            : base(balance)
        {
            Name = name;
            TotalAmountSpent = total;
        }

        public new string Name
        {
            get { return name; }
            set
            {
                if (string.IsNullOrEmpty(value))
                    throw new ArgumentException("Name cannot be empty");
                name = value;
            }
        }

        public int TotalAmountSpent
        {
            get { return totalAmountSpent; }
            set
            {
                if (value < 0)
                    throw new ArgumentException("Total cannot be negative");
                if (value > 0 && value < 50000)
                    throw new ArgumentException("Total must be at least 50000 for a regular
customer");
                totalAmountSpent = value;
            }
        }

        public override (string Name, int PersonalDiscount) GetDiscount(Goods goods)
        {
            int PersonalDiscount = TotalAmountSpent / 1000;
            if (PersonalDiscount > 15)
                PersonalDiscount = 15;
            if (PersonalDiscount > goods.MaxDiscount)
                PersonalDiscount = goods.MaxDiscount;

            return (Name, PersonalDiscount);
        }

        public override bool Buy(Goods goods)
        {
            var (customerName, discount) = GetDiscount(goods);

            int finalPrice = goods.Price - (goods.Price * discount / 100);

```

```

if (Balance >= finalPrice)
{
    TotalAmountSpent += finalPrice;
    Balance -= finalPrice;
    return true;
}
else {
    return false;
}

}

public override string ToString()
{
    return $"Постійний покупець: {Name}, Баланс: {Balance}, Витрачено усього:
{TotalAmountSpent}";
}

}
}

```

Лістинг Б.10 – Клас Purchase, аркуш 2

```

using System;
using System.Collections.Generic;
using System.Security.Cryptography;

namespace Kursach.Classes
{
    public class Session
    {
        private Shop currentShop;
        private Customer currentCustomer;
        private int securityLevel;

        private List<Shop> shopList = AddStandartShops();
        private List<Customer> customerList = AddStandartCustomers();
        private static Random random = new Random();

        public Session()
        {
            CurrentShop = null;
            CurrentCustomer = null;
            SecurityLevel = 0;
        }

        public Shop CurrentShop {
            get { return currentShop; }
            set { currentShop = value; }
        }

        public Customer CurrentCustomer
        {
            get { return currentCustomer; }
            set { currentCustomer = value; }
        }

        public int SecurityLevel
        {
            get { return securityLevel; }
            set { securityLevel = value; }
        }
    }
}

```

Лістинг Б.11 – Клас Session, аркуш 1

```

public List<Customer> CustomerList {
    get { return customerList; }
}
public List<Shop> ShopList
{
    get { return shopList; }
}

public static List<Customer> AddStandartCustomers()
{
    var list = new List<Customer>();
    int CustomersCount = random.Next(1, 4);
    var usedIndexes = new HashSet<int>();

    list.Add(new Customer());

    for (int i = 0; i < CustomersCount; i++)
    {
        int index = random.Next(StandartCustomer.Count);
        list.Add(StandartCustomer[index]);
    }
    return list;
}

public static List<Shop> AddStandartShops()
{
    var list = new List<Shop>();
    int ShopsCount = random.Next(3, 5);
    var usedIndexes = new HashSet<int>();
    for (int i = 0; i < ShopsCount; i++)
    {
        int index = random.Next(StandartShop.Count);

        while(!usedIndexes.Add(index))
        {
            index = random.Next(StandartShop.Count);
        }
        list.Add(StandartShop[index]);
    }
    return list;
}

public void Reset()
{
    CurrentShop = null;
    CurrentCustomer = null;
    SecurityLevel = 0;
}

public static readonly List<Customer> StandartCustomer = new List<Customer>
{
    new RegularCustomer(0, "Andrii", 175000),
    new RegularCustomer(0, "Maksym", 98000),
    new RegularCustomer(0, "Oleksandr", 250000),
    new RegularCustomer(0, "Dmytro", 55000),
    new RegularCustomer(0, "Serhii", 310000),
    new RegularCustomer(0, "Vladyslav", 67000),
    new RegularCustomer(0, "Yaroslav", 145000),
    new RegularCustomer(0, "Bohdan", 89000)
};
public static readonly List<Shop> StandartShop = new List<Shop>
{

```

```

        new Shop("23 Shevchenka Ave", 20, "Petrenko"),
        new Shop("78 Prymorska St", 35, "Koval"),
        new Shop("10 Deribasivska St", 50, "Shevchenko"),
        new Shop("5 Fontanska Rd", 15, "Bondar"),
        new Shop("120 Lustdorfska Rd", 40, "Melnyk"),
        new Shop("44 Balkivska St", 60, "Tkachenko"),
        new Shop("9 Hretska Sq", 25, "Ivanenko")
    };
}
}

```

Лістинг Б.11 – Клас Session, аркуш 3

```

using System;
using System.Collections.Generic;

namespace Kursach.Classes
{
    public class Shop
    {
        private string address;
        private int storageCapacity;
        private string managerName;
        private List<Goods> goodsList = AddDefaultGoods();
        private List<Purchase> history = new List<Purchase>();
        private static Random random = new Random();

        public Shop() : this("N/A", 0, "N/A") { }
        public Shop(string address, int storageCapacity, string managerName)
        {
            Address = address;
            StorageCapacity = storageCapacity;
            ManagerName = managerName;
        }

        public string Address
        {
            get { return address; }
            set
            {
                if (string.IsNullOrEmpty(value))
                    throw new ArgumentException("Address cannot be empty");
                address = value;
            }
        }

        public int StorageCapacity
        {
            get { return storageCapacity; }
            set
            {
                if (value < 0)
                    throw new ArgumentException("Storage capacity cannot be negative");
                storageCapacity = value;
            }
        }

        public string ManagerName
        {
            get { return managerName; }
        }
    }
}

```

Лістинг Б.12 – Клас Shop, аркуш 1


```

        set
        {
            if (string.IsNullOrEmpty(value))
                throw new ArgumentException("Manager name cannot be empty");
            managerName = value;
        }
    }

    public List<Goods> GoodsList
    {
        get { return goodsList; }
    }

    public List<Purchase> History
    {
        get { return history; }
    }

    public bool HasGoods()
    {
        return GoodsList != null && GoodsList.Count > 0;
    }

    public void PrintGoods()
    {
        Console.Clear();
        Console.WriteLine($"Товари в магазині за адресою: {Address}");

        if (GoodsList == null) {
            Console.WriteLine("У магазині не має товарів");
            Console.WriteLine("Для продовження натисніть будь яку кнопку");
            Console.ReadKey();
            return;
        }

        for (int i = 0; i < GoodsList.Count; i++)
        {
            Console.WriteLine($"{i + 1} {GoodsList[i]}");
        }
        Console.WriteLine("Щоб купити товар, введіть його номер");
        Console.WriteLine("Для виходу введіть 0");
    }

    public void AddPurchase(Customer buyer, Goods product, int price) {
        History.Add(new Purchase(buyer, product, price));
    }

    public bool CheckManagerName(string name)
    {
        return name == ManagerName;
    }

    public void PrintHistory()
    {
        Console.WriteLine($"Історія покупок в магазині за адресою: {Address}");
        if (History.Count == 0)
        {
            Console.WriteLine("Історія покупок порожня");
        }
        else
        {

```

```

        foreach (var purchase in History)
        {
            Console.WriteLine(purchase);
        }
    }

    public string ToString(int securityLevel)
    {
        if (securityLevel >= 2)
            return $"Магазин: Адреса: {Address} \nЄмність складу: {StorageCapacity}
одиниць \nКількість товарів: {GoodsList.Count} \nНомер гарячої лінії: (049) 949-23-32
\n\nГрафік роботи: \nPн-Пт 9:00-20:00 \nCб 10:00-18:00 \nНеділя вихідний";
        else
        {
            return $"Магазин: Адреса: {Address} \nНомер гарячої лінії: (049) 949-23-32
\n\nГрафік роботи: \nPн-Пт 9:00-20:00 \nCб 10:00-18:00 \nНеділя вихідний";
        }
    }

    public static List<Goods> AddDefaultGoods()
    {
        var list = new List<Goods>();
        int itemCount = random.Next(3,7);
        var usedIndexes = new HashSet<int>();

        for (int i = 0; i < itemCount; i++)
        {
            int index = random.Next(StandartCatalog.Count);

            while (!usedIndexes.Add(index))
            {
                index = random.Next(StandartCatalog.Count);
            }
            list.Add(StandartCatalog[index]);
        }
        return list;
    }

    public static readonly List<Goods> StandartCatalog = new List<Goods>
    {
        new VacuumCleaner("Bosch", "BGC05AAA1", 3699, 10, 78, 6, 700),
        new VacuumCleaner("Samsung", "VC07M2110SR", 4200, 10, 80, 7, 750),
        new VacuumCleaner("Philips", "FC9332", 4999, 10, 79, 6, 900),
        new VacuumCleaner("Xiaomi", "Mi Vacuum 1C", 6500, 10, 82, 6, 1200),
        new VacuumCleaner("Rowenta", "R03731", 5200, 10, 77, 5, 800),

        new Camera("Canon", "PowerShot SX40 HS", 15999, 15, 12),
        new Camera("Nikon", "Coolpix B500", 13999, 15, 16),
        new Camera("Sony", "DSC-H300", 12500, 15, 20),
        new Camera("Panasonic", "Lumix DMC-FZ82", 18500, 15, 18),
        new Camera("Kodak", "PixPro AZ401", 11000, 15, 16),
    }

```

```

        new DSLR("Sony", "Alpha A100", 23500, 10, 10, true, 4000),
        new DSLR("Canon", "EOS 2000D", 24999, 10, 24, true, 5000),
        new DSLR("Nikon", "D3500", 26000, 10, 24, true, 4500),
        new DSLR("Pentax", "K-70", 28000, 10, 24, true, 4200),
        new DSLR("Canon", "EOS 90D", 42000, 10, 32, true, 6000),

        new Computer("Custom", "WorkStation Pro", 94500, 10, "i9-14900K", 64, "RTX
4080"),

        new Computer("HP", "OMEN 45L", 85000, 10, "i7-14700K", 32, "RTX 4070"),
        new Computer("Dell", "XPS Desktop", 78000, 10, "i7-13700", 32, "RTX 4060"),
        new Computer("Lenovo", "Legion Tower 7", 90000, 10, "i9-13900", 64, "RTX
4080"),

        new Computer("Acer", "Predator Orion", 82000, 10, "i7-13700KF", 32, "RTX
4070"),

        new Laptop("ASUS", "Vivobook 16X", 42000, 10, "i7-12700H", 16, "RTX 3050",
1.7, 10),
        new Laptop("Lenovo", "IdeaPad 5", 35000, 10, "i5-1235U", 16, "Intel Iris
Xe", 1.6, 12),
        new Laptop("HP", "Pavilion 15", 38000, 10, "i5-12450H", 16, "RTX 2050",
1.8, 11),
        new Laptop("Acer", "Swift 3", 33000, 10, "i5-1135G7", 8, "Intel Iris Xe",
1.2, 13),
        new Laptop("MSI", "Modern 15", 45000, 10, "i7-1260P", 16, "MX550", 1.6, 10)
    };
}
}

```

Лістинг Б.12 – Клас Shop, аркуш 4